CHAPTER 4

LINKED LIST

Definition

- It is a **list or collection of data items** that can be stored in scattered locations (positions) in memory by establishing link between the items.
- To store data in scattered locations in memory we have to **make link** between one data item to another.
- So, each data item or element must have **two parts**: one is data part and another is **link (pointer) part**.

Definition [cont..]

- Each data item of a linked list is called a node. Data part contains (holds) actual data (information) and the link part points to the next node of the list.
- To locate the list an external pointer is used that points the first node of the list.
- The link part of the last node will not point any node. That means it will be *null*.
- This type of list is called linear (one way) linked list or simply linked list.

A single node of linked list

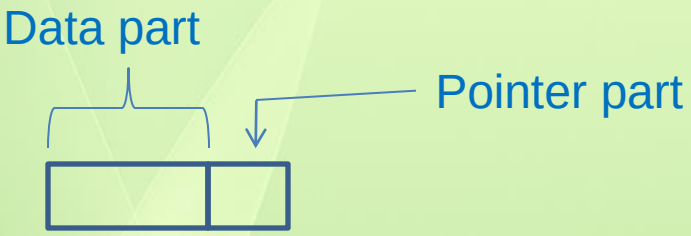


Fig: 4.1 Graphical representation of a node

A node can be declared/define using code as follows:

```
struct node
 {
int data;
node *next;
}
```

Variable for data part

Variable for pointer part
(A pointer that will point next node)

Graphical Representation

Linked list:

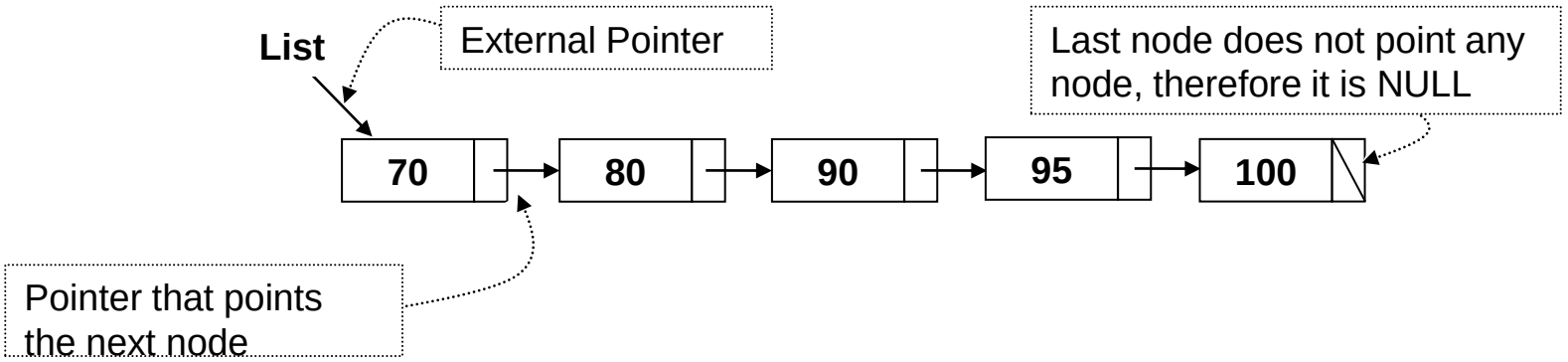


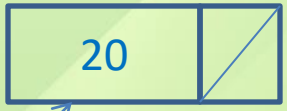
Figure 4.2: Graphical representation of a linear linked list

Storing data in a node

1. Node declaration:



```
struct node
 {
 int data;
 node *next;
 }
```



nptr

2. Storing data:

```
node *nptr;            // declara a pointer variable
nptr= new(node);       // allocate space (memory)
nptr->data= 20;        // insert data to data part
nptr->next = NULL;     // assign NULL to ponter part
```

Create a new node

1. Node declaration:

```
struct node
        {
        int data;
        node *next;
        };
```

2. Declare variable (pointer type) that point to the node:

```
node *nptr;
```

3. Allocate memory for new node:

```
nptr = new (node);
```

4. Enter value:

```
nptr→data = item; //item is a variable
nptr→next = NULL;
```

A Linked List Creation Process

1. Create an empty linked list.

   (That means, the external pointer will be *null*.)

2. Create a new node.

   (The data part of the new node will contain data (information) and the pointer part will contain *null*.)

3. The external pointer will point the new node.

   (At this point only one node in the list.)

4. Create another new node and include the node to the linked list.

5. **Repeat the process** to include any other node.

   (Thus we can create a linked list.)

Linked List Creation Process

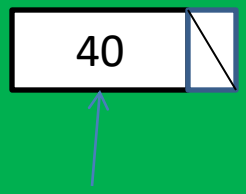
1) Create an empty list, the pointer list will point to Null

```
list = NULL;
```



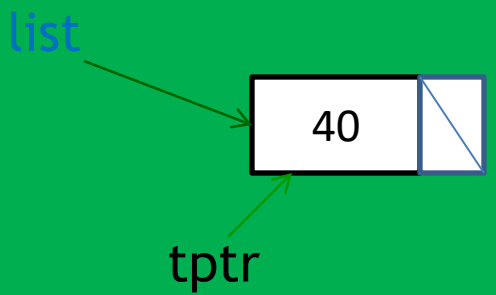
2) Create a new node with data

```
nptr = new (node);
nptr ->data = 40;
nptr -> next = NULL;
```



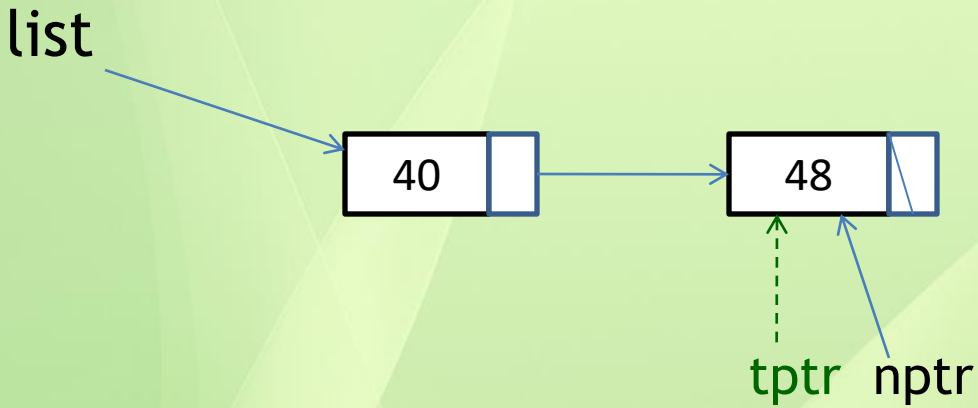
3) Include the node to the list

```
list = nptr;
tprt= nptr;
```



Here tptr is temporary pointer used to make link between existing list and new node.

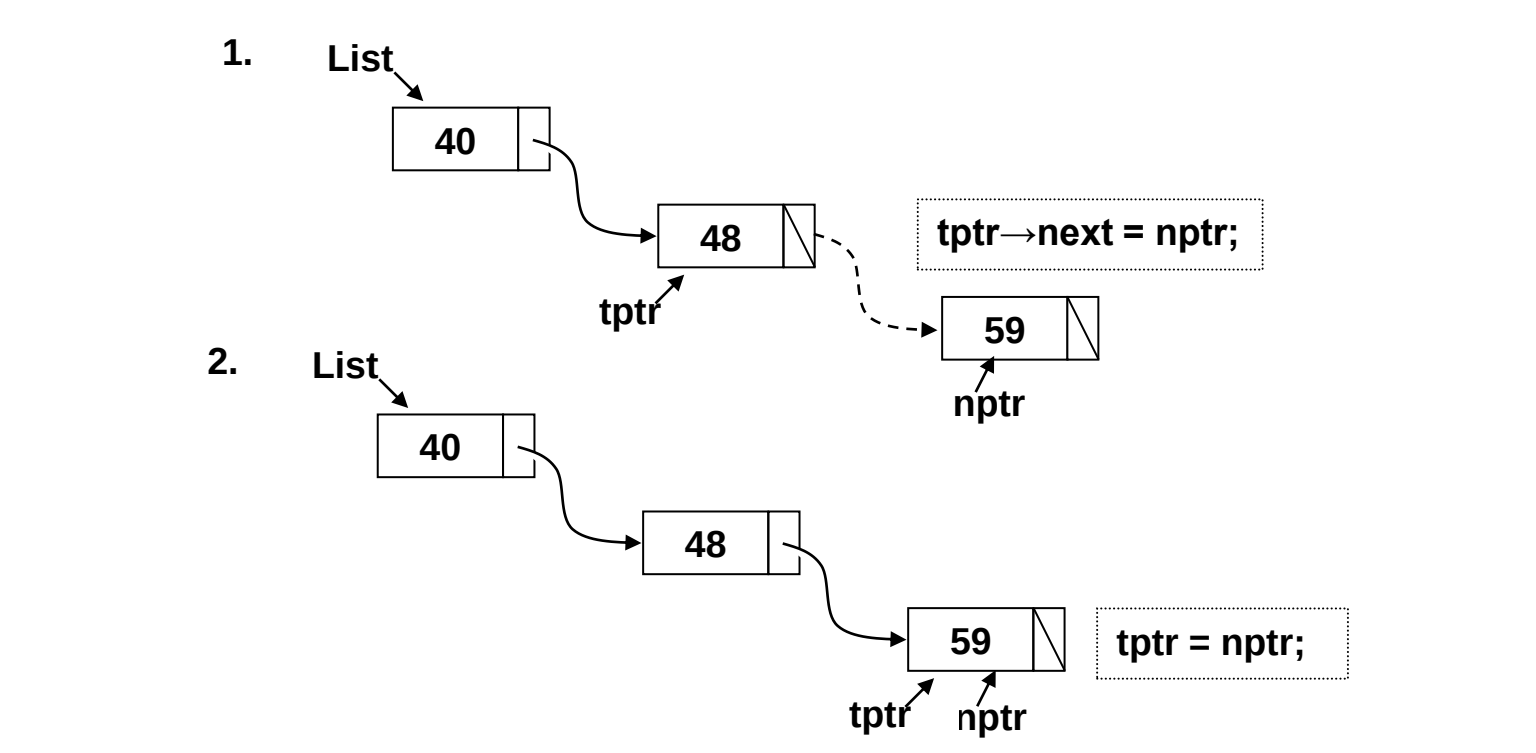
Linked List Creation Process (New Node Inclusion)




```
tptr ->next = nptr;
tptr = nptr;
```

Addition of node to a linked list (Graphical view)

- We shall enter data in ascending order and create the list.



- Figure 4.2: A pictorial view of addition of items to a linked list

Algorithm to create a linked list (pseudocode)

1. Declare node and pointers :

```
a. struct node
        {
    int data;
    node *next;
        };
b. node *list, *tptr, *nptr;
```

2. Create an empty list:

```
list = NULL;
```

3. Create a new node:

```
nptr = new (node);
nptr→data = item;
nptr→next = NULL;
```

4. **Make link between the linked list and the new node:**

```
if (list = = NULL)
    {
      list = nptr;
      tptr = nptr;
     }
else
    {
    tptr→next = nptr;
    tptr = nptr;
}
```

5. Repeat Step 3 and 4 necessary times to add more nodes.

6. Output linked list.

Program to create a linked list

```
void main()
 {
  struct node
 {
     int data;
    node *next;
 };
int i,n,item;
node *nptr, *tptr, *list; // Necessary pointers
 list=NULL;                // Create an empty list
cout<<"Enter number of nodes:";
cin>>n;
```

Program [Cont..]

```
cout<<"Enter data for node with space:";
for (i=1;i<=n;++i)
{
        cin>>item;
    nptr=new (node);
    nptr->data=item;
nptr->next=NULL;
if (list==NULL)  //List is empty
    {
    list=nptr;  //Include first node
    tptr=nptr;
    }
```

```
//There is/are node(s)

else
  {

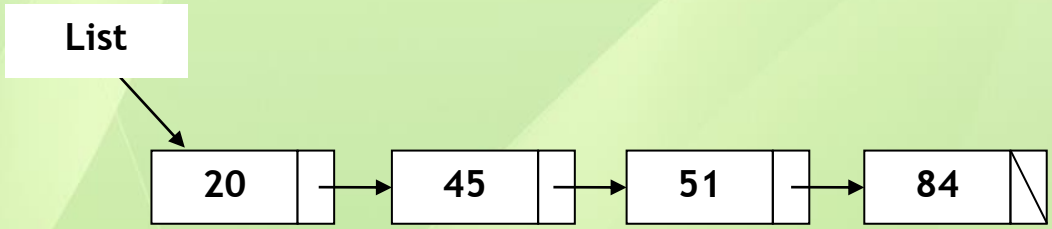
     tptr->next=nptr;
      tptr=nptr;
  }
      } // end of for loop
```

Program [Cont..]

```
//Display data
    tptr=list;
    for (i=1; i<=n;++i)
     {
    cout<<endl;
    cout<<tptr->data;
    tptr=tptr->next;
     cout<< " ";
     }
     cout<<endl;
     cout<<endl;
     }
```

Search or Locate a node of a linked list

Given a linked list, we have to **find out a node** whose value is known such as 51.



Locate or Search a node

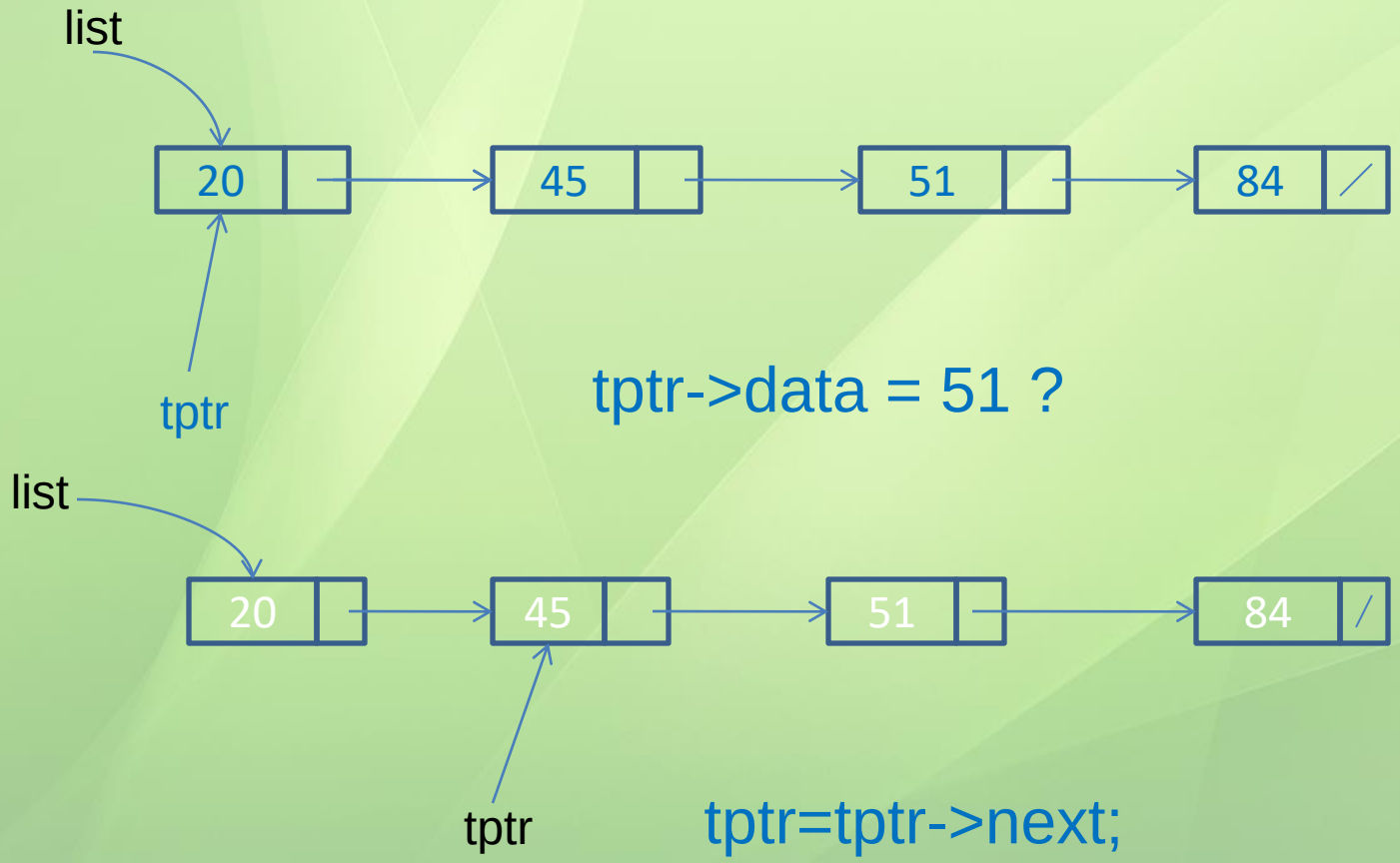
Problem 5.1: Find out the item 51 from above linked list. All items in the list were stored in ascending order.

To locate the node we have to traverse the list using a pointer.

a) We have to use a temporary pointer to traverse the list.

b) At each node we shall compare and check whether we have found the node or not.

Locate a node of a linked list (Graphical view)



Algorithm to search a node from a linked list (pseudocode)

1. Define a variable (item) and a pointer (**tptr**)
2. Input the value to be located:
   item = 51;
3. Search the item:
   tptr = list;
   while (tptr→data !=item and tptr->next! = NULL)
   {
   tptr = tptr→next;
   }
4. Output:
   if (tptr→data == item);
   print "**FOUND**"
   else print "**NOT FOUND**"

Insert a node into a list

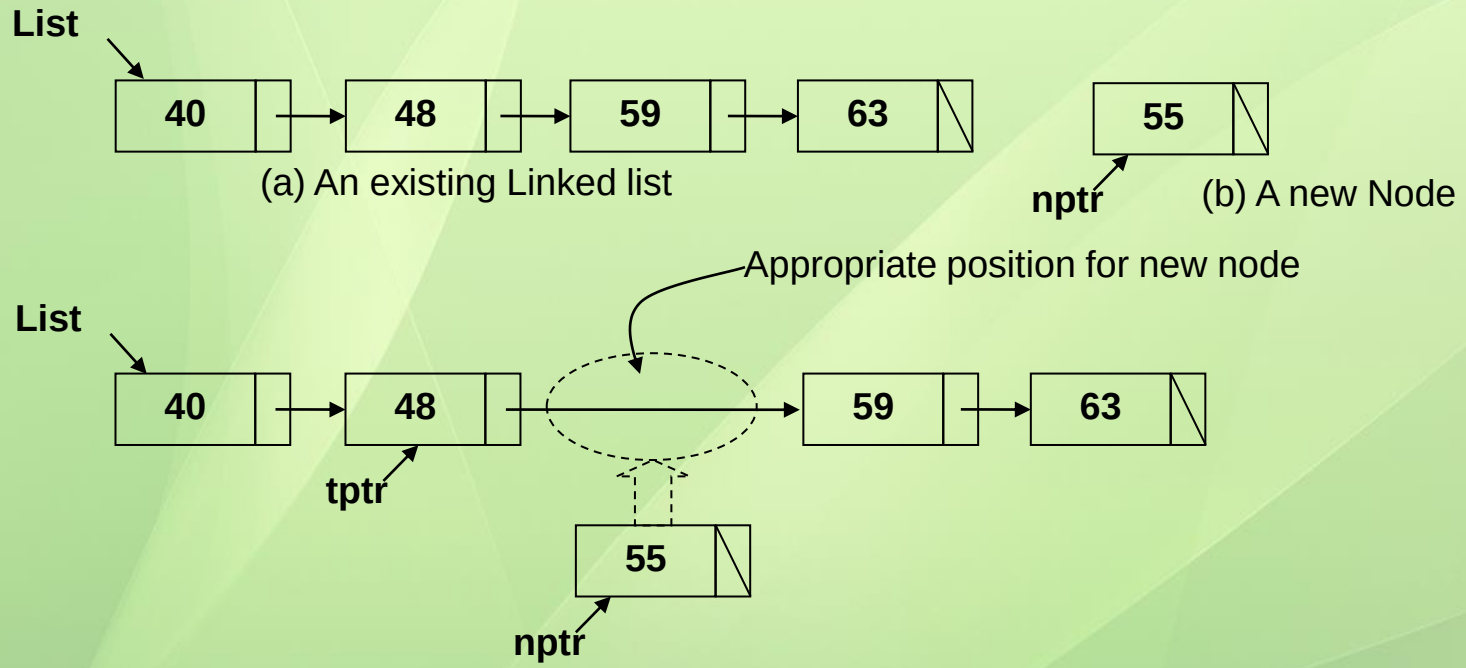
Here, we shall consider insertion of a node after the first node or before the last node and the data of the list are arranged in ascending order.

Here we have to perform two major tasks:

1. Locate (find out) the node after which the new node will be inserted.

2. Insert the node by making link.

Locate the position to insert (Graphical view)

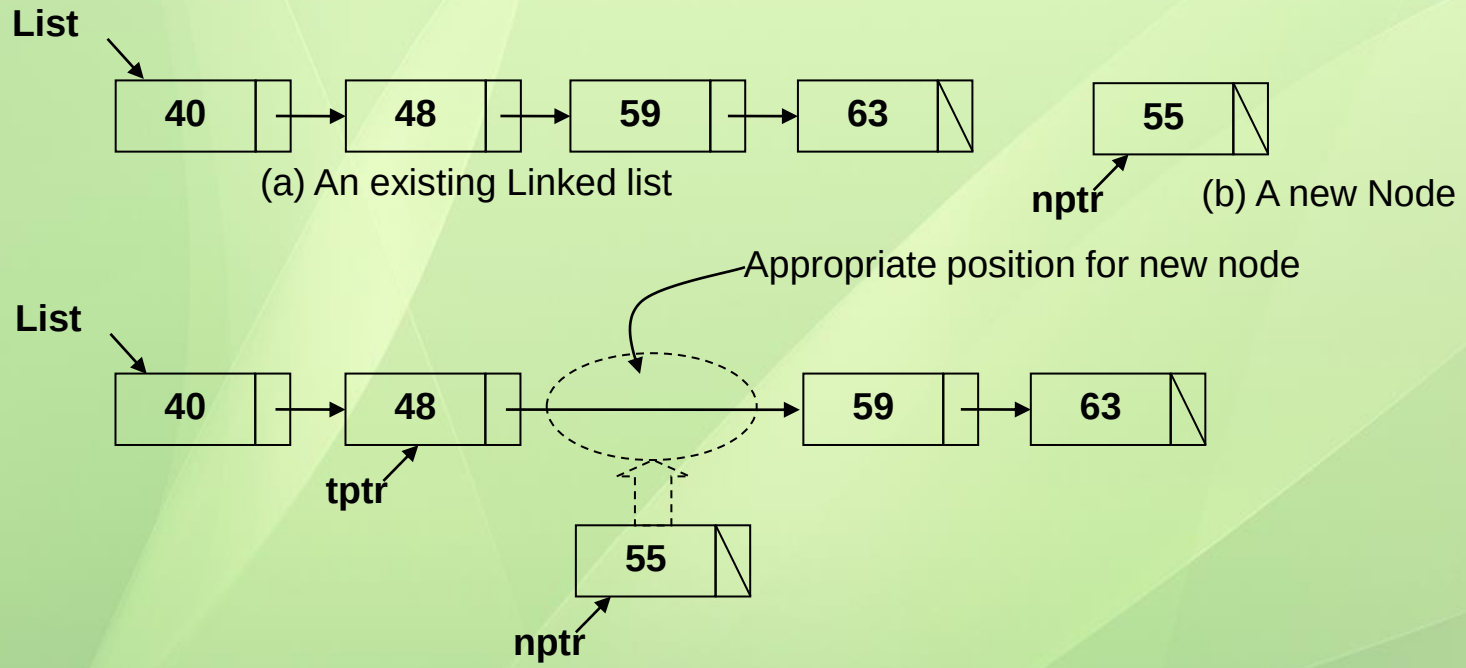


Locate the proper position for insertion

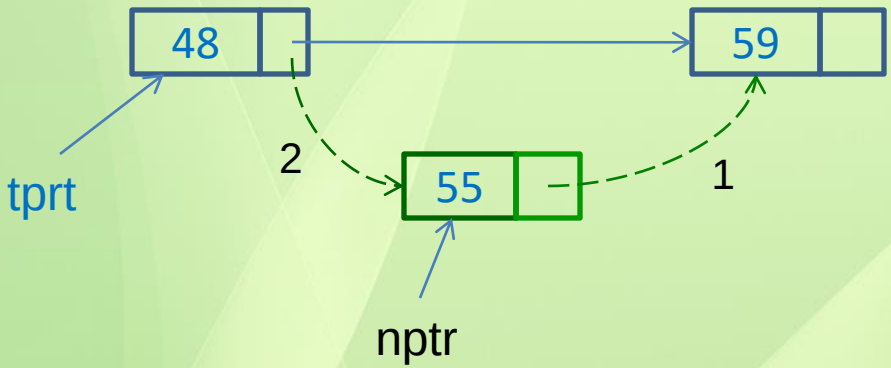
The steps to locate the position:

1. Assign the value of external pointer to a temporary pointer (tptr = list).
2. Compare the value of the next node with the value of the new node.
3. Traverse the temporary pointer until we find a greater node value than the value of the new node (tptr = tptr->next).

Locate the position to insert (Graphical view revisit)



Node insertion in between two nodes



Insert the node by making link. The steps are:

1. Point the next node by the new node.

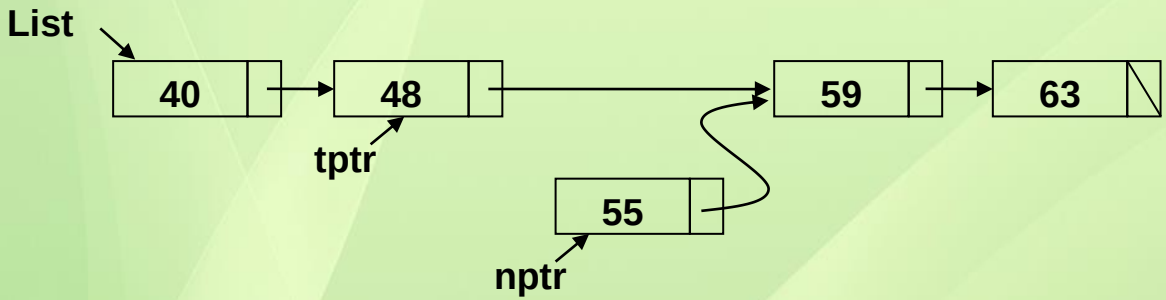
   (nptr->next = tptr->next).

2. Point the new node by the previous node.

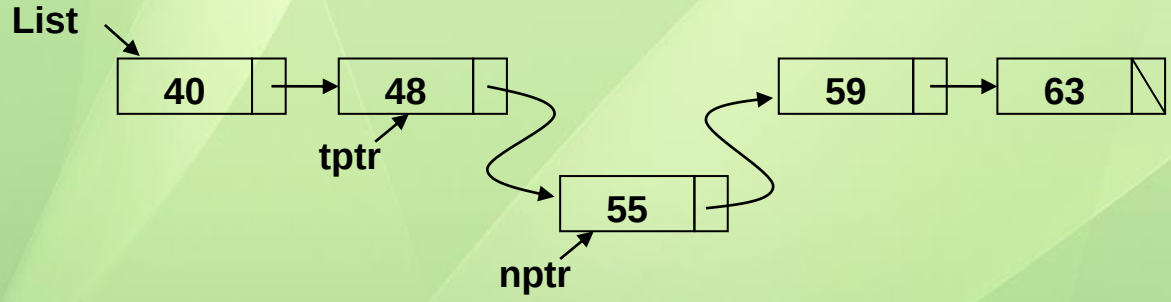
   (tptr->next = nptr).

3. We have got an updated linked list.

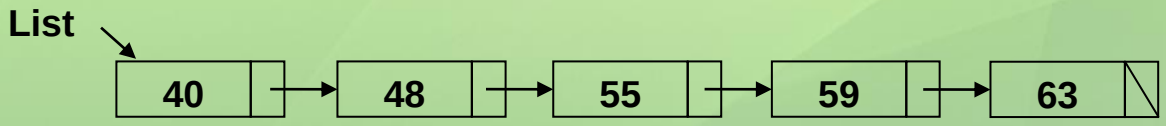
Insert the node into a list (Graphical view)



(d) Making link between the new node and the node after the *tptr.*



(e) Making link between tptr and the new node



(f): Updated List

Algorithm to insert a node into a linked list (pseodocode)

Here, we shall consider insertion after the first node or before the last node.

1. Input linked list (we have to use an existing list)
2. Declare pointers (list, tptr, nptr)
3. Create a new node:

```
nptr = new (node);
nptr→data = item;
nptr→next = NULL;
```

4. Locate the appropriate position for the new node

```
while (tptr→next→data < tptr→data)
    {
    tptr = tptr→next;
    }
```

5. Insert the new node at appropriate position (by linking previous and next node);

```
nptr→next = tptr→next;
tptr→next = nptr;
```

6. Output :updated linked list

Delete a node from a linked list

To delete a particular node from a linked list,

Three major tasks to be performed.

1. Find out or search the node to be deleted.
2. Establish the necessary link.
3. Delete the node.

Locate the node to be deleted

The steps to locate the node:

1. Use a temporary pointer and assign it the value of the external node (tprt= list).
2. Compare the value of the node pointed by the temporary pointer and the value of the node to be deleted.
3. If the first node is the target node, then we found the node. Otherwise,
4. Assign the value of the temporary pointer to another pointer or save the value (pptr=tptr).
5. Traverse the temporary node until we locate or find the node (tptr = tptr->next).

Establishing link before deletion

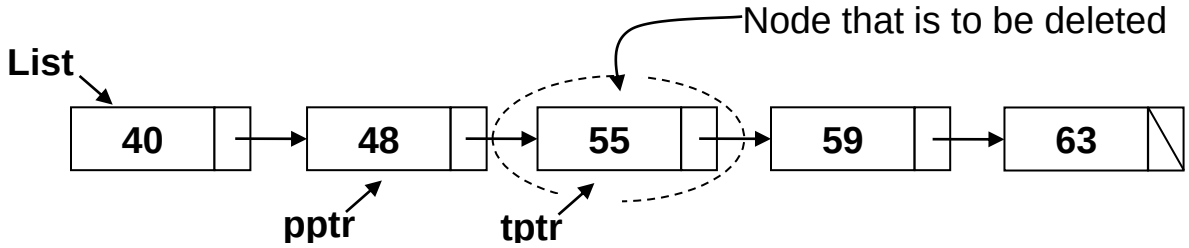
Making link between the nodes. The steps are:

1. The previous node will point the next node of the node to be deleted (pptr->next = tptr->next).
2. Delete the node.

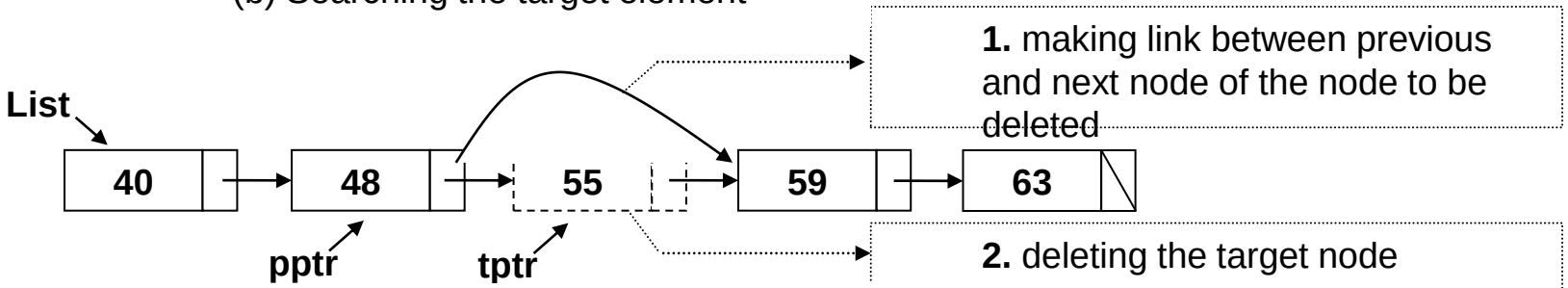
Deletion of a particular node (Graphical view)



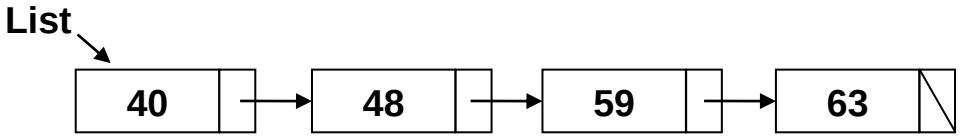
(a) An existing Linked list



(b) Searching the target element



(c) Deleting the target node



(d) Updated List

Algorithm to delete a node from a linked list (pseudocode).

Here we shall not consider the deletion process of the first node and the last node of the list.

1. Declare pointers (list, tptr, pptr)
2. Input linked list and the item (that is to be deleted)
3. Search the item to be deleted in the list:

```
tptr = list;
while (tptr→data != item)
        {pptr = tptr;
         tptr = tptr→next; }
```

4. Delete the node:

[Make link between previous and next node of the node that is to be deleted and delete the target node]

```
pptr→next = tptr→next;
delete (tptr);
```

5. Output: updated linked lists

DOUBLY LINKED LIST

OR

TWO WAY LINKED LIST

Doubly Linked List

Definition:

A doubly or two way linked list is a list where each node has three parts.

- One is link or pointer to the previous (backward) node and
- One is data part to hold the data and
- Another is link or pointer to the following (forward) node.

  - There is an external pointer to the first node of the list. It is also called two-way linked list.

Doubly Linked List (Graphical representation)

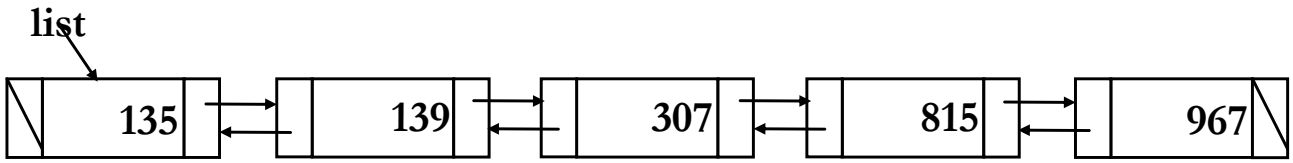
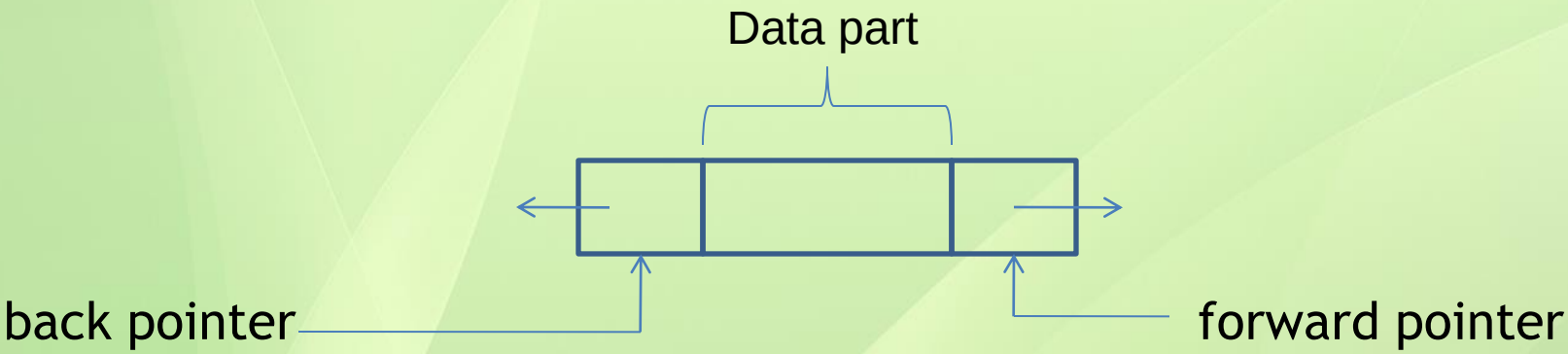


Figure 4.5: Graphical representation of a doubly linked list

A node of doubly linked list



Node declaration:

```
struct node:
  {
  node *back;   //back pointer
  int data;
  node *next;    //forward pointer
   }
```

Create a node (pseudocode)

1. Declare a node:

```
struct node
    {
    node *back;
    int data;
    node *next;
    };
```

2. Create a node:

```
node *nptr;
nptr = new (node);
nptr→back = NULL;
nptr→data = item;   // item is a variable
nptr→next = NULL;
```

Creation Process of a Doubly Linked List

1. Create empty linked list (list =NULL).
2. Create a new node with data (see slide# 33).
3. Include the node to the list by making link.
4. Create another new node with data and include the node to the list.
5. Repeate the process.

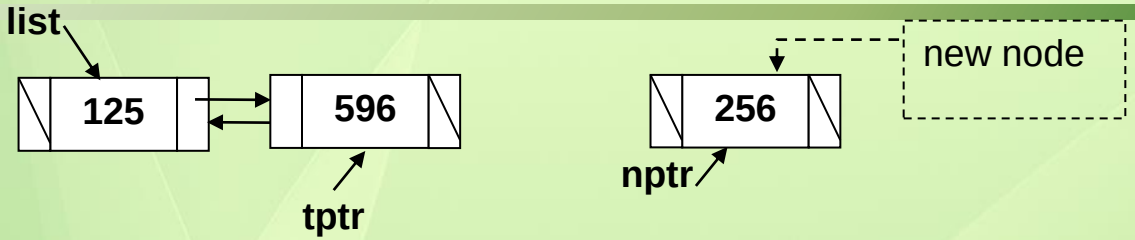
Include a new node to the list

The steps to include a new node to a doubly linked list:

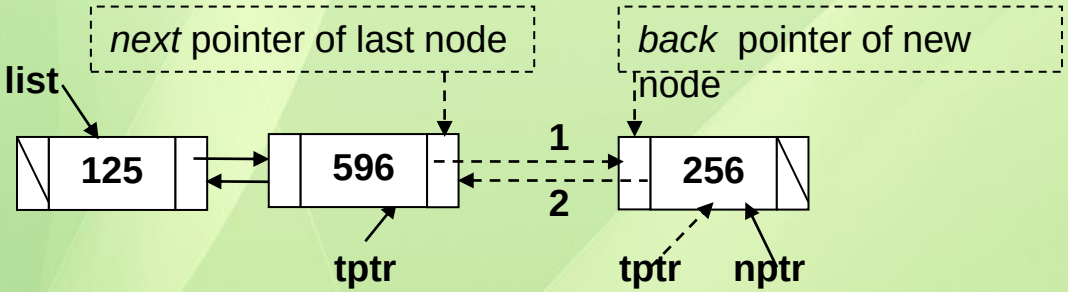
1. The ***next*** pointer of the last node will point the new node (tptr->next = nptr).

2. The ***back*** pointer of the new node will point the last node of the list (nptr->back = tptr).

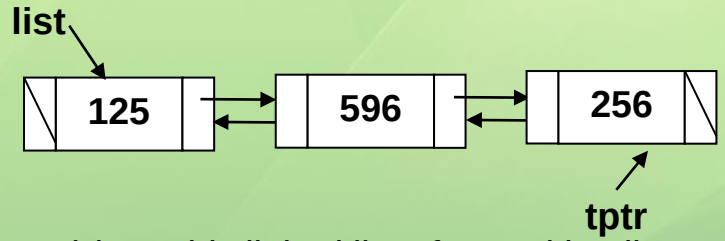
Create a Doubly linked list (Graphical view)



(a) A doubly linked list and new node



(b) Linking new node and the last node of the list



(c) Doubly linked list after making list

Figure 4.6: Creation of doubly linked list (Pictorial View

Algorithm to create a doubly linked list (pseudocode)

1. Declare node and pointers:

```
a. struct node
        {
        node *back;
        int data;
        node *next;
        }
b. node *list, *tptr;
```

2. Create an empty list:

```
list = NULL;
```

3. Create a new node:

```
node *nptr;
nptr = new (node);
nptr→back = NULL;
nptr→data = item;
nptr→next = NULL;
```

4. Make link between the last node of the list and the new node:

```
if (list = NULL)
        {
        list = nptr;
        tptr = nptr;
        }
else
       {
        tptr→next = nptr;
        nptr→back = tptr;
        tptr = nptr;
        }
```

5. Repeat step 3 and step 4 necessary times.

6. Output a doubly linked list.

Insert a node into a doubly linked list

To Insert a node into a linked list we have to perform two major tasks:

1. Search or locate the proper position in the list where we have to insert the node.
2. Establish the link between the new node and existing nodes.

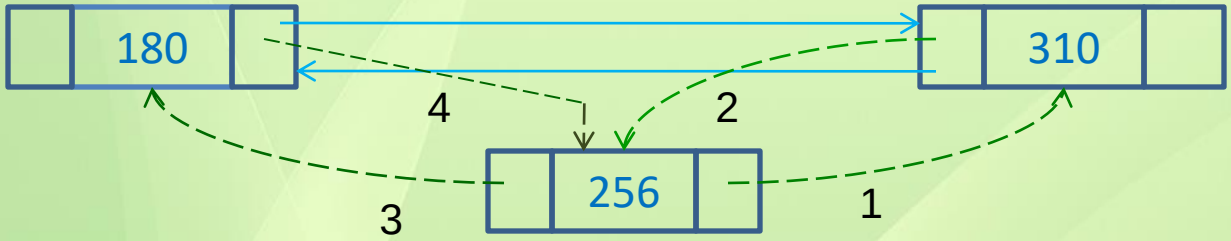
Node insertion [Contin..]

The steps to search the proper position for insertion:

1. Use temporary pointer (tptr) to find the position.
2. Start from the first node of the list (tptr = list).
3. Compare the data of the new node with the data of the next node to the temporary pointer.
4. If the data of the next node is greater, then we have found the position. Otherwise,
5. Traverse the temporary pointer (tprt = tptr->next).
6. Repeat the step 4 to step 6 until we find the position.

Node Insertion (Cont..]

Making link in case of insertion of new node in between two nodes:

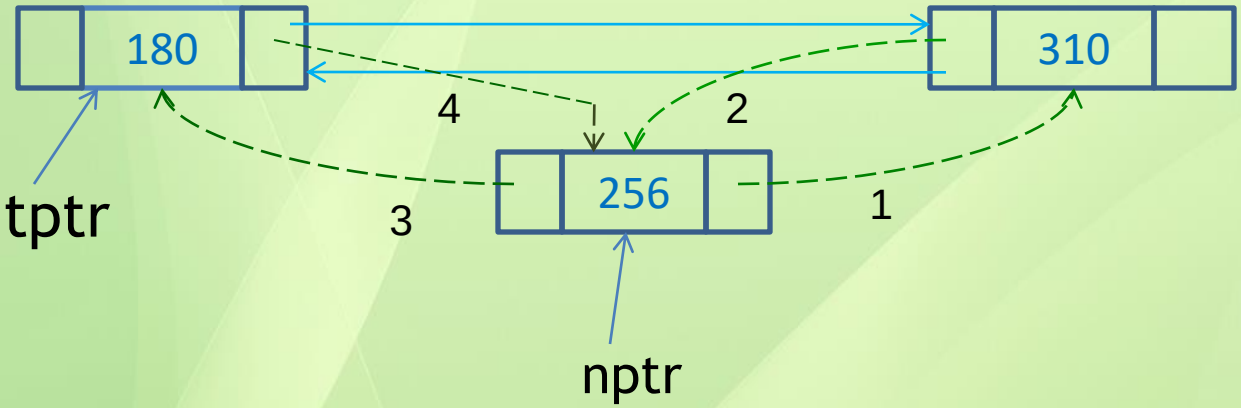


The following are the steps to establish link:

1. Make link from the new node to the next node (of the new node's position).
2. Make link from the next node to the new node.
3. Make link from the new node to the previous node (of the new node's position).
4. Make link from the previous node to the new nodeh

Node Insertion (Cont..]

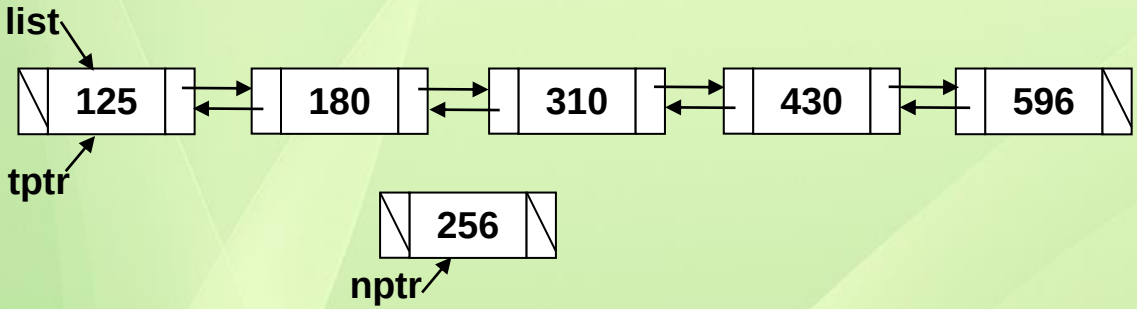
Making link in case of insertion of new node in between two nodes:



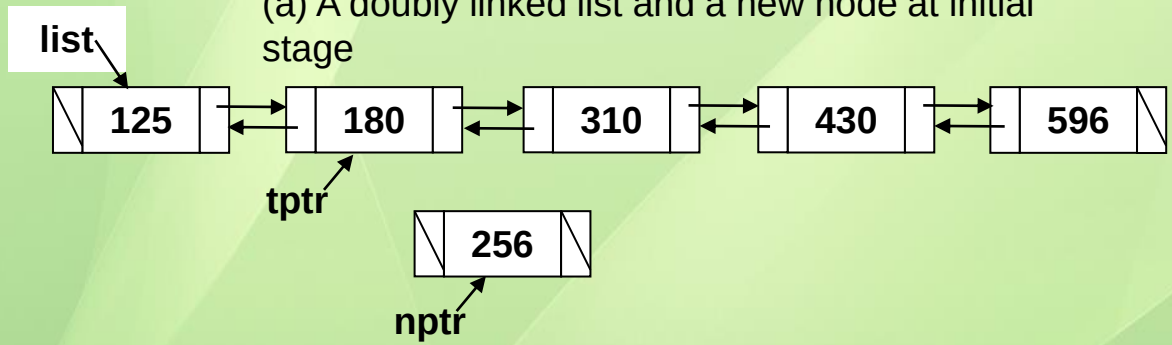
The following are the codes to establish link:

1. nptr ->next = tptr ->next;
2. tptr ->next ->back = nptr;
3. nptr ->back = tptr;
4. tptr ->next = nptr;

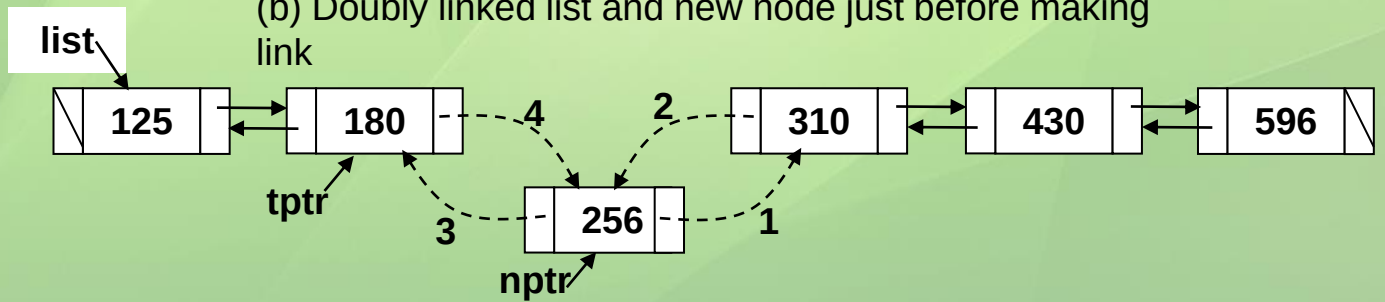
Insertion of a node into a doubly linked list (Graphical view)



(a) A doubly linked list and a new node at initial stage



(b) Doubly linked list and new node just before making link



(c) Doubly linked list and new node after making link

Algorithm to insert a node into doubly linked list (pseucode)

Here we have considered the case of insertion in between two nodes:

1. Input a doubly link list;
2. Declare necessary pointers (list, nptr, tptr);
3. Create a new node with nptr pointer;
4. Locate the position of the node before or after which the new node will be inserted.
5. Insert the node in appropriate position:
   a. If temporary pointer (tptr) is at the first node and insert the new node before the first node:

```
nptr→next = tptr;
tptr→back = nptr;
list = nptr;
```

Algorithm to insert a node into doubly linked list [contd.]

b. If temporary pointer (tptr) is not at the first or at the last node and insert in between two nodes:

```
nptr→next = tptr→next;
tptr→next→back = nptr;
nptr→back = tptr;
tptr→next = nptr;
```

c. If temporary pointer is at the last node and insert after last node:

```
tptr→next = nptr;
nptr→back = tptr;
```

6. Output: Updated linked list.

Delete a node from a doubly linked list

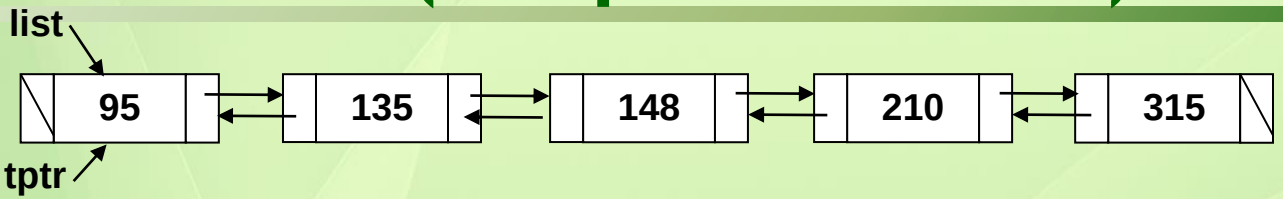
Three major tasks to be performed to delete a node:

1. Locate the node to be deleted.

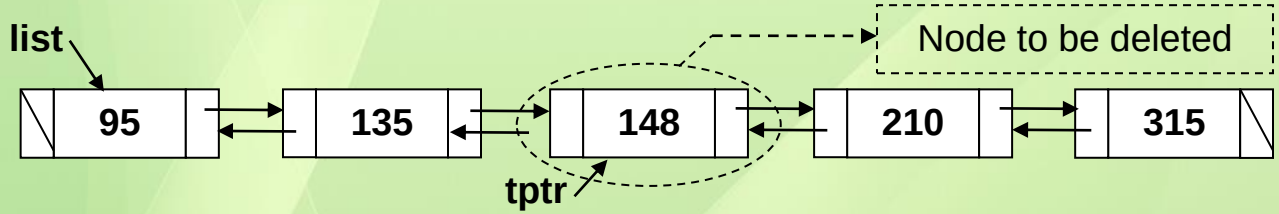
2. Make necessary link.

3. Delete the node.

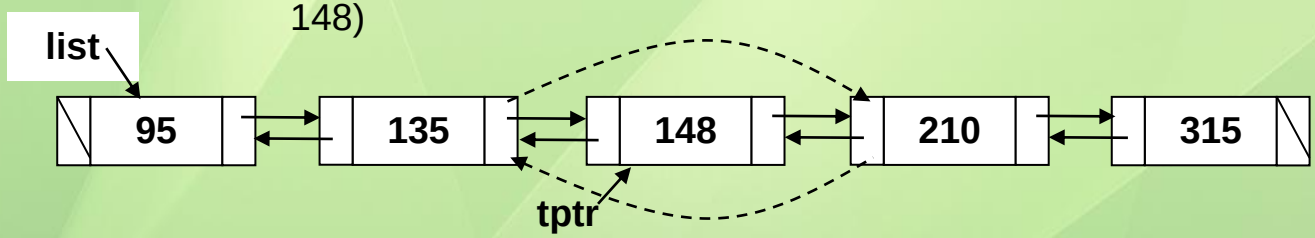
Deletion of a node from a doubly link list (Graphical view)



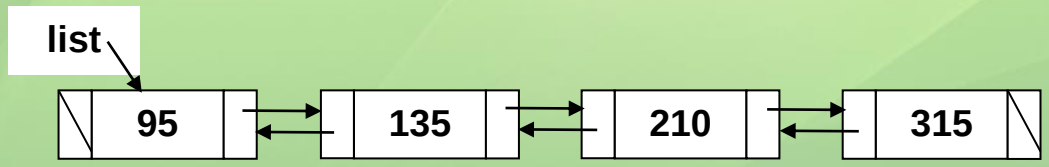
(a) Doubly linked list at initial stage



(b) Doubly linked list just before deletion of a node (with value 148)



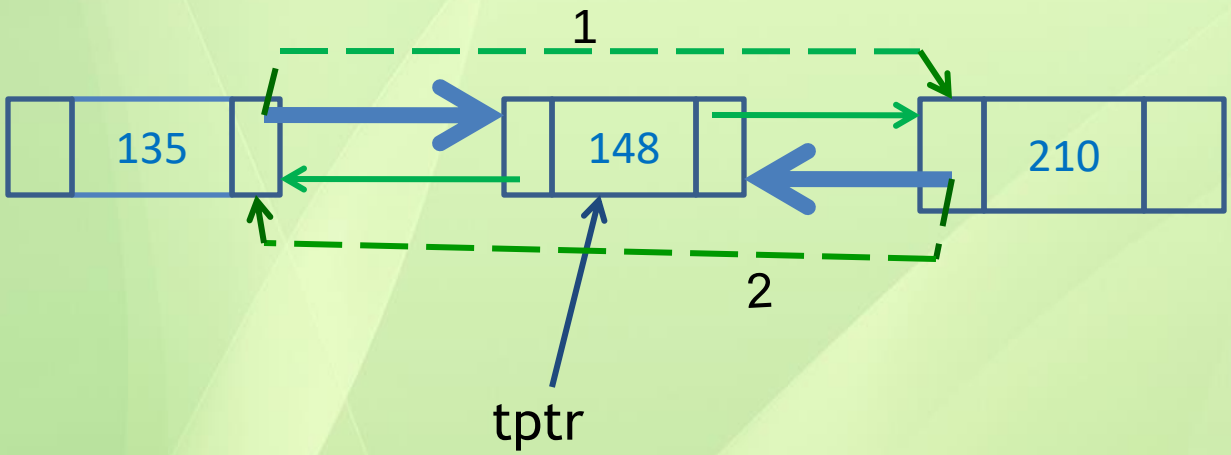
(c) Doubly linked list after making link and before deletion of the node



(d) Updated doubly linked list after deletion of target node

- Figure 4.8: Deletion of a node from a doubly linked list (pictorial view)

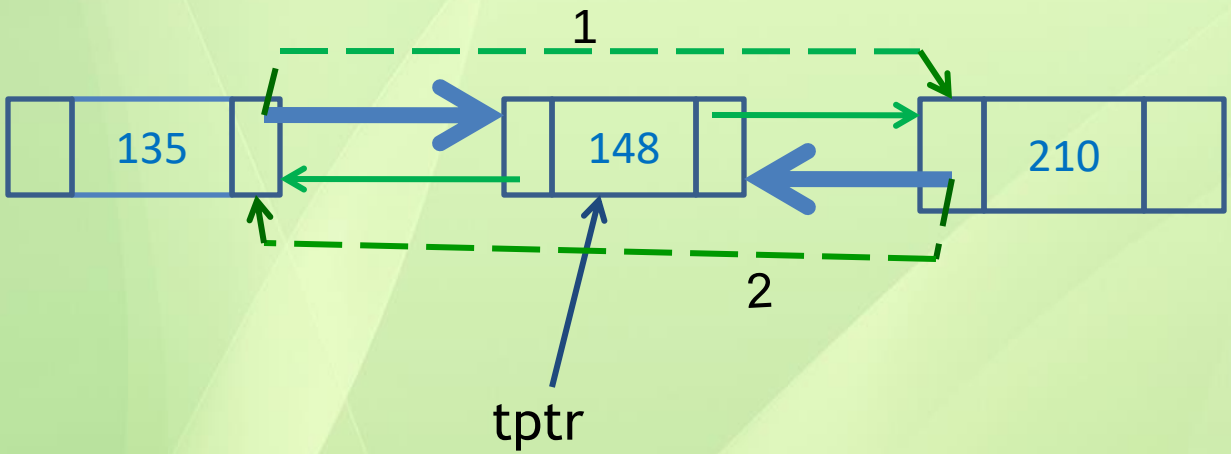
Node Deletion (Cont..]



The following are the steps to establish link:

1. Make link from the previous node to the next node (of the node to be deleted).
2. Make link from the next node to the previous node (of the node to be deleted).

Node Deletion (Cont..]



Establishing link using code:

1. tptr->back->next = tptr->next;
2. tptr-next->back = tptr->back;

Algorithm to delete a node from a doubly linked list (pseudocode)

1. Input a doubly link list and item (value of the node to be deleted);
2. Declare necessary pointers (list, tptr);
3. Locate the node to be deleted:

```
        tptr = list;
    while (tptr→next != NULL)
        {
        if (tptr→data = = item) break;
        tptr = tptr→next;
        }
```

4. Make link:
   a. If the node to be deleted is the first node of the list:

```
        list = tptr→next;
        list→back = NULL;
        [if there is only one node in the list then list = NULL]
```

Algorithm to delete a node [Cont..]

b. If the node to be deleted is not the last node:

```
tptr→back→next = tptr→next;
tptr→next→back = tptr→back;
```

c. If the node to be deleted is the last node:

```
tptr→back→next = NULL;
```

5. Delete the target node:

```
delete (tptr);
```

6. Output: Updated doubly linked list.

Circular Linked List

- A circular linked list is a list where each node has two parts; one is data part to hold the data and another is pointer part that points the next node and the last node's pointer points the first node of the list.
- And there is an external pointer to the list that points the first node.

Circular Linked List [contd.]

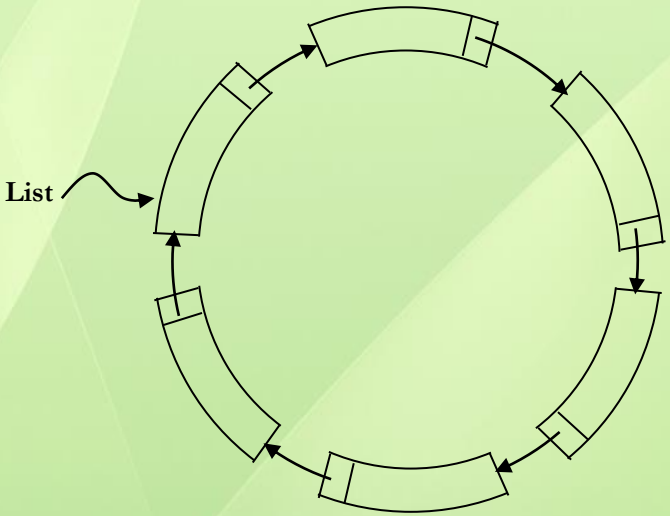
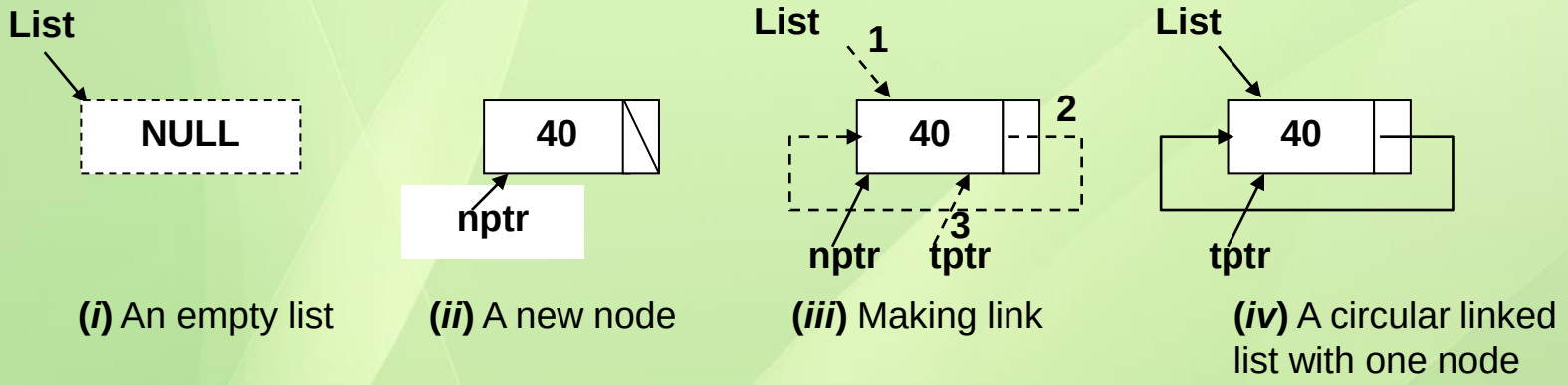


Figure 4.9: Pictorial view of a circular linked list (a circular diagram)

Create a circular linked list

- Creation process of a circular linked list is similar to the creation process of a linear linked list which is already discussed.
- Here we have to make link between the last node and the first node which will create a circle (linked list as a circle).

Create a circular linked list (Graphical view)



(*i*) An empty list (*ii*) A new node (*iii*) Making link (*iv*) A circular linked list with one node

Figure 4.11: Linking process of the first node of a circular linked list

Create a circular linked list [contd.]

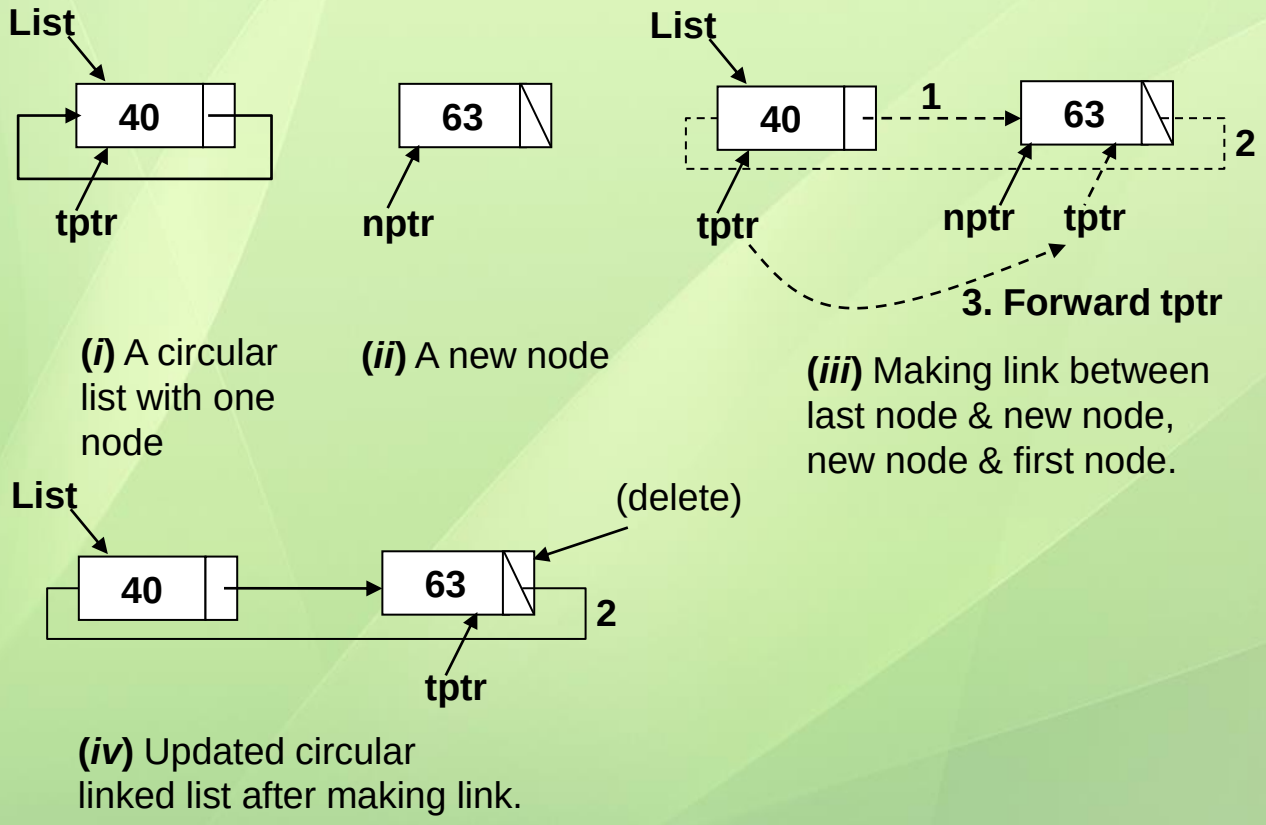


Figure 4.12: Creation of a circular linked list with more than one node

Algorithm to create a circular linked list (pseudocode)

1. Declare node and pointers (list, tptr, nptr)
2. Create an empty inked list
   list = NULL;
3. Create a new node with data:
   nptr = new (node);
   nptr→data = item;
   nptr→next = NULL;
4. Make link between new node and the link list:
   if (list = NULL)
   {
   list = nptr;
   nptr→next = list; //for circular link
   tptr = nptr;
   }
   else
   {
   tptr→next = nptr;
   nptr→next = list; //Circular link
   tptr = nptr;
   }
5. Output a circular linked list.

Difference between array and linked list

1. An array is a finite set of same type of data items (elements). In other words, it is a collection of homogeneous data items. The elements of an array are stored in successive memory locations. Any element of an array is referred by array name and index number (subscript).

   Whereas, linked list is a list or collection of data items stored in scattered memory locations. Each data item has two parts. One is data part and another is link (pointer) part. Each data item of a linked list is called node. Data part holds actual data (information) and the link part points to the next node of the list. To locate the list or the 1st node of the list, an external pointer is used. The link part of the last node will not point any node. That means it will be null.

Difference between array and linked list [contd.]

2. Array implementation depends on size and it results in wastage of memory space. On the other hand, linked list does not depend on size.
3. Types of array are one dimensional, two dimensional etc. and types of linked list are linear, doubly, circular etc.
4. An element of an array can be accessed directly and access time is fixed as well as efficient. On the other hand, a node of a linked list can not be accessed directly and access time is linear and not so efficient.
5. Array is a static data structure and a linked list is a dynamic data structure.

Comparison of operations using array and linked list.

Array	Linked list
Element access is fast if index is known.	Element access is slow.
Insertion and deletion operations are slow.	Insertion and deletion operations are fast.
Element search is slow.	Element search is slow.

This is the end of linked list

THANK YOU.